Saturday, March 14, 2026 7:36 AM

GeeksforGeeks

Introduction to Doubly Linked List

```
# Node Class
    class Node:
        def __init__(self, data):   # data -> value stored in node
            self.data = data
            self.next = None
            self.prev = None
'''
class Solution:
    def constructDLL(self, arr):
        if not arr:
            return None


        head=Node(arr[0])
        prev=head

        for i in range(1, len(arr)):
            new_node=Node(arr[i])
            prev.next= new_node
            new_node.prev= prev
            prev=new_node

        return head
```

arr = [1, 2, 3, 4, 5]

[prev | data | next]

Step-1 Create Head Node

We will create the Node by arr[0]

We will create the first node from arr[0]

head
↓

[None | 1 | None]

prev pointer variable points to head

prev = head

Step 2 Insert 2

new node
↓

[None | 2 | None]

Connect Pointers

prev.next = new_node

new_node.prev = prev

None <- 1 <-> 2 -> None

↑
head

Pointer Structure

1.next -> 2
2.prev -> 1

Step 3 -> Insert 3

Create Node 3

Link Nodes
2.next -> 3
3.prev -> 2

None <- 1 <-> 2 <-> 3 -> None
↑
head

move Pointer
prev = 3

Step-4 Insert 4

Create Node 4

3.next -> 4
4.prev -> 3

None <- 1 <-> 2 <-> 3 <-> 4 -> None
↑
head

prev = 4

Step 5 -> Insert 5

Create node 5

4.next -> 5
5.prev -> 4

Final Structure

Node <- 1 <-> 2 <-> 3 <-> 4 <-> 5 -> None
↑
head

Final Memory Structure

Node 1	prev = None	data = 1	next = 2
Node 2	prev = 1	data = 2	next = 3
Node 3	prev = 2	data = 3	next = 4
Node 4	prev = 3	data = 4	next = 5
Node 5	prev = 4	data = 5	next = None

Time Complexity O(n)
Space Complexity O(n)

Insertions at Doubly Linked List

```
class Solution:
    def insertAtPos(self, head, p, x):
        newNode=Node(x)
        curr=head
```

```python
        for _ in range(p):
            curr=curr.next

        newNode.next=curr.next
        newNode.prev= curr

        if curr.next:
            curr.next.prev=newNode
        curr.next= newNode
        return head
```

List :- 2 <-> 4 <-> 5

p = 2

n = 6

Insert 6 after the 2nd node

Step 1 Initial Doubly Linked List

Null <- [2] <-> [4] <-> [5] -> Null

0 1 2

head = 2

Traverse to p-th node

Iteration 1

curr = 4

Iteration 2

curr = 5

Now,

curr → [5]

Create New Node

newNode = Node(6)

[6]

prev = None

next = None

Update Pointers

newNode = curr.next

Since, curr.next = NULL

[6]

prev = None

next = NULL

Connect newNode.prev

newNode.next

Since, curr.next = NULL

[6]
prev = None
next = NULL

Connect newNode.prev

[5] <-> [6]

fix next node (skip because NULL)
if curr.next.prev = NewNode
Not Executed

Attach New Node

curr.next = newNode

Final connection
[5] <-> [6]

Final Doubly Linked List

NULL <- [2] <-> [4] <-> [5] <-> [6] -> NULL

0 1 2 3

Pointer Structure

2.next -> 4
4.prev -> 2

4.next -> 5
5. prev -> 4

5.next -> 6
6. prev -> 5

6. next = NULL